

Constitutional Grounding vs. Dynamic Skill Loading: An Empirical Evaluation of Autonomous Agents in Terminal Environments on Terminal-Bench 2.1

CARB Empirical Evaluation Team
Artifacts and code available in the benchmark repository

September 2026

Abstract

Autonomous software engineering agents operating in terminal environments are increasingly tasked with complex debugging, systems administration, and algorithmic workflows. A central architectural consideration in agent design is how guidance is structured: embedding invariant engineering principles directly within the initial context versus dynamically loading specialized procedural skills on demand. In this work, we compare the behavior of three substantially different agent-guidance configurations across all 89 tasks of the official Terminal-Bench 2.1 benchmark (267 isolated container evaluations) using Gemini 3.6 Flash High within a unified evaluation harness: Supreme (an invariant constitutional framework), a dynamic skill-loading system based on Superpowers by obra (obra/superpowers), and an unprompted baseline. Across the full benchmark, Supreme achieved the highest task completion (61/89, 68.5% vs. 58/89, 65.2% for Superpowers and 55/89, 61.8% for Baseline; 70.9% vs. 67.4% vs. 64.0% after adjusting for upstream container defects), the lowest aggregate wall-clock runtime (15.15h vs. 18.30h and 16.92h), and the lowest effective paid cost per successful task (\$0.489, approximately 3.4% below Superpowers at \$0.506). While aggregate accuracy differences across the 89 tasks are not statistically distinguishable under paired McNemar tests ($p > 0.05$) and instruction content remains coupled with delivery mechanisms, the configurations exhibit distinct behavioral and economic profiles. On Hard tasks ($N = 30$), Supreme demonstrated an increase in provider-reported deliberation tokens (+66.0%, from 17.0k to 28.3k thinking tokens/task) alongside a directional pass-rate lead (66.7% vs. 53.3%; paired McNemar $p = 0.22$, exact binomial $p = 0.22$). Transcripts reveal stark operational failure modes: in open-ended combinatorial tasks (e.g., Task #88), unconstrained dynamic skill exploration coincided with extensive script-generation loops, writing 116 exploratory files and consuming 3.51 million tokens ($11.7\times$ the tokens of Supreme’s 300k-token solve). Furthermore, context-growth accounting indicates that frequent on-demand documentation ingestion accumulated 411.3 million cache read tokens (+115.4 million over baseline). We provide complete execution transcripts, evaluation harness configurations, and cryptographic audit ledgers to facilitate independent replication.

1 Introduction

The deployment of frontier large language models as autonomous agents in real-world software engineering and systems administration requires navigating complex, stateful terminal environments [3, 6]. Unlike single-turn question-answering or isolated code generation benchmarks, terminal-grounded tasks demand sustained multi-turn reasoning, file system traversal, compiler diagnostics, and iterative error recovery [8, 11].

As agent systems scale in complexity, a central architectural question is how operational capabilities, procedural guidance, and behavioral constraints should be structured:

1. Dynamic Progressive Skill Loading: Exemplified by modular agent frameworks such as Superpowers by obra [10], this design aims to minimize initial prompt length by exposing a lightweight catalog of capabilities.

The agent dynamically identifies relevant skills (e.g., test-driven development, systematic debugging, git worktrees) and loads detailed procedural documentation on demand via file-reading tools.

2. Constitutional Static Grounding: Embodying the Supreme paradigm [1], this architecture directly embeds invariant engineering axioms (evidence-over-assumption, surgical editing boundaries, hierarchical decomposition, and loop management) permanently within the static system prompt header.

While on-demand skill discovery is conceptually appealing for conserving initial prompt tokens, its empirical behavior across long-horizon terminal tasks remains largely unexamined under controlled experimental conditions. Earlier exploratory evaluations on simpler 50-task subsets (e.g., CARB-v2) encountered severe benchmark ceiling effects: the underlying foundation model solved 92% of tasks across all configurations, yielding an in-

distinguishable result (Baseline 94%, Supreme 94%, Superpowers 92%; paired McNemar $p = 1.0$). This ceiling demonstrated that standard, easily memorized tasks mask critical architectural failure modes. To provide genuine discriminative depth, this work evaluates the complete, unmodified 89-task corpus of Terminal-Bench 2.1, encompassing heavy systems administration, multi-stage compilation, and low-level kernel emulation.

In this work, we compare the behavior of three substantially different agent-guidance configurations across all 89 tasks of the official Terminal-Bench 2.1 benchmark (267 total container evaluations) using Gemini 3.6 Flash High within a unified evaluation harness: Supreme, a dynamic progressive skill configuration based on Superpowers by obra, and an unprompted baseline.

Research question: How do two substantially different agent-guidance configurations—constitutional static grounding and dynamic progressive skill loading—compare in task completion, execution economics, and behavioral trajectory when operating under the same foundation model, evaluation harness, and benchmark corpus?

Critically, the evaluated configurations differ in both guidance delivery mechanism (static header vs. dynamic on-demand loading) and instructional semantic content (constitutional axioms vs. procedural skill libraries). This study evaluates the complete, real-world configurations rather than performing controlled ablations that isolate delivery mechanism from semantic content. The implications for architectural attribution should be interpreted accordingly.

Our investigation yields four primary contributions:

- **Hard-Task Deliberation Patterns:** We observe that under constitutional static grounding, provider-reported deliberation tokens scale by +66.0% on Hard tasks, coinciding with a directional pass-rate advantage (66.7% vs. 53.3%; paired McNemar $p = 0.22$) over dynamic skill loading, where extensive documentation injection increased context volume.
- **Characterization of Exploratory Parameter Thrashing:** We document an operational failure mode in combinatorial search problems (exemplified by Task #88), where dynamic skill guidance coincided with recursive script generation (116 exploratory scripts) and an 11.7 $\times$ token inflation (3.51M tokens) relative to a targeted hypothesis-driven solve.
- **Multi-Turn Context Accounting and Cache Telemetry:** We quantify token distribution across 267 executions, finding that on-demand skill injection accumulated 411.3 million cache read tokens (+115.4M over baseline), illustrating how dynamic file loading is associated with context accumulation patterns.
- **Polyglot Domain Breakdown and the Esoteric Anomaly:** Across 5 technical engineering stacks, we evaluate domain sensitivity, observing that

while Supreme shows directional advantages in Low-Level Systems and SysAdmin, unprompted Baseline achieves high pass rates in archaic languages (COBOL, Scheme: 85.7%), where procedural refactoring recipes introduced unnecessary file-churn overhead.

2 Related Work

Autonomous Software Engineering Benchmarks. Early LLM coding benchmarks, such as HumanEval and MBPP, evaluated isolated algorithmic functions via synthetic unit tests. SWE-bench [3] advanced the field by introducing real-world GitHub issues, requiring agents to navigate large repositories and generate unified patch diffs. However, SWE-bench remains largely text-driven and static. Terminal-Bench 2.1 [6] extends agent evaluation into interactive, multi-container Linux environments, demanding full shell competency, polyglot package compilation, daemon orchestration, and dynamic runtime debugging across diverse technical stacks.

Agent Prompt Architectures. The foundational ReAct framework [11] interleaves chain-of-thought reasoning traces with action execution. Subsequent research has explored dynamic capability adaptation, such as Toolformer [5], which trains models to invoke APIs on demand. In open-source agent ecosystems, dynamic skill extensions have gained widespread adoption. Superpowers by obra [10] formalizes progressive disclosure, enabling agents to dynamically discover and read structured procedural skills from disk, ostensibly minimizing initial token consumption.

Constitutional AI and Guardrails. Constitutional AI [1] demonstrated that foundation models can be aligned through explicit self-governing principles rather than unconstrained RLHF. Supreme v1.0 operationalizes constitutional principles for production systems engineering, embedding operational constraints—such as surgical diff boundaries, hypothesis-driven debugging, and anti-premature-closure audits—directly into the invariant prompt context to prevent speculative code mutation and runaway tool loops.

Prompt Caching and Multi-Turn Attention Physics. Frontier model APIs have introduced automated prompt caching [4, 8] to dramatically reduce latency and billing on repetitive context prefixes. However, prompt caching algorithms rely strictly on contiguous prefix matching. When autonomous agents inject dynamic skill documents, large file dumps, or fluctuating tool schemas mid-session, the prefix cache is invalidated, negating anticipated token economies. Our work provides the first large-scale empirical audit of this dynamic in multi-turn environments.

3 Agent System Architectures

To strictly isolate the impact of prompt and systems architecture from foundation model variance, all three paradigms were evaluated using the identical underlying foundation model engine (gemini-3.6-flash-high) at zero temperature ($\tau = 0.0$), with identical tool schemas for shell execution, file reading, file editing, and process management.

3.1 Supreme (v1.0): Constitutional Static Grounding

Supreme v1.0 structures agent behavior through a static, invariant constitutional framework permanently positioned in the system prompt header. It consists of five core operational modules:

- **Engineering Constitution:** Invariant behavioral axioms prioritizing directly verifiable evidence over model assumption, user intent preservation, and proportional scrutiny scaling with action blast radius.
- **Operating Protocol:** A decision framework enforcing four sequential execution phases: intent extraction, hierarchical decomposition, surgical code modification, and multi-signal verification.
- **Surgical Editing Mandate:** Explicit diff constraints requiring agents to justify line changes, use targeted line-replacement tools (`replace_file_content`), and strictly avoid opportunistic or cosmetic file rewrites.
- **Anti-Premature-Closure:** A mandatory verification forcing function requiring agents to ask: "If this implementation were falsely claiming completion, what plausible omission was overlooked?"
- **Loop Management Protocol:** Hard limits on consecutive failing commands and strict prohibitions against passive background task polling loops.

Under this architecture, the system prompt prefix of length $L_{\text{const}} \approx 4,800$ tokens remains strictly invariant across all turns $t \in [1, T]$, ensuring deterministic prefix cache hits on every interaction turn.

3.2 Superpowers by obra: Dynamic Progressive Skill Loading

Superpowers by obra [10] implements two-tier progressive capability disclosure. The initial system prompt contains a compact index summarizing 14 distinct engineering skills:

1. using-superpowers (Meta-discovery & catalog dispatch)
2. systematic-debugging (Root-cause tracing and defense-in-depth)

3. test-driven-development (Red-Green-Refactor TDD cycle)
4. writing-plans (Hierarchical multi-phase execution planning)
5. executing-plans (Systematic plan fulfillment and checkpointing)
6. verification-before-completion (Pre-flight validation gates)
7. subagent-driven-development (Isolated task delegation)
8. requesting-code-review (Pre-commit review requests)
9. receiving-code-review (Critique integration and refactoring)
10. finishing-a-development-branch (Merge sanitation and branch cleanup)
11. dispatching-parallel-agents (Concurrent sub-agent dispatch)
12. using-git-worktrees (Isolated filesystem branching)
13. brainstorming (Design exploration and architecture ideation)
14. writing-skills (Dynamic procedural skill creation)

When an agent determines that a skill is required, it dynamically invokes file-reading tools to ingest the corresponding SKILL.md and nested reference documents into the conversational history. While reducing the initial prompt to $\approx 1,200$ tokens, this on-demand injection inserts reference text directly into the active dialogue turns.

3.3 Baseline: Unprompted Control

The Baseline configuration provides the raw LLM engine with execution capabilities and tool declarations, but omits all auxiliary system instructions, behavioral protocols, and skill catalogs. It serves as the empirical control to measure the baseline competence of the underlying model.

3.4 Methodological Scope: Structure vs. Content

We emphasize that Supreme and Superpowers differ along two dimensions simultaneously: structural delivery (static invariant prompt header vs. on-demand dynamic disk retrieval) and semantic instruction content (constitutional axioms and surgical diff boundaries vs. procedural engineering recipes such as TDD and worktrees). Consequently, the empirical results presented in this work reflect the overall behavior of the complete configurations.

Controlled ablations (e.g., statically injecting Superpowers documentation, or applying constitutional loop controls to dynamic skill loading) will be valuable in future work to cleanly decouple delivery mechanisms from prompt semantics.

4 Experimental Setup

4.1 Benchmark Corpus: Terminal-Bench 2.1

We evaluate across the complete 89-task corpus of Terminal-Bench 2.1 [6]. To analyze real-world polyglot versatility beyond standard Python scripting, we classify all 89 tasks across five distinct technical engineering stacks:

- Low-Level & Systems Engineering ($N = 20$): C, C++, x86/ARM/MIPS Assembly, Redcode, Rust, and OCaml compiler runtimes.
- SysAdmin, OS & DevOps ($N = 15$): Linux kernel parameters, QEMU hypervisor orchestration, Git object reconstruction, OpenSSH, and Bind9 DNS.
- Software Engineering ($N = 26$): Python applications, Cython extensions, complex data pipelines, and multi-file refactors.
- Esoteric & Symbolic Programming ($N = 14$): Scheme metacircular evaluators, COBOL business logic, LaTeX typesetting, Stan probabilistic modeling, and SPARQL ontology queries.
- Machine Learning & Robotics ($N = 14$): PyTorch distributed parallelism, HuggingFace transformers, FastText embeddings, and MuJoCo XML physics modeling.

4.2 Evaluation Isolation and Anti-Leakage Protocol

To ensure benchmark validity and prevent evaluation leakage during task execution, the evaluation harness enforces strict multi-container isolation:

1. **Pristine Container Provisioning:** An isolated Docker container is spawned from the official benchmark image with bounded memory and CPU allocations.
2. **Evaluation Isolation:** During active agent problem-solving, the ground-truth verification suite (`/tests`) is completely absent from the container filesystem. Across all 267 transcripts, agents executed 104 exploratory shell searches attempting to locate verification files (e.g., `find / -name "*test*"`); all 104 attempts returned empty results. This confirms that agents did not access ground-truth tests during execution, though it does not preclude pre-training data exposure.

3. **Per-Task Timeout Enforcement:** Each task operates under its official per-task timeout T_{limit} specified in its benchmark configuration (`task.toml`). Across the corpus, timeouts range from 360s to 3,600s with a median of 900s ($N = 48$ tasks). Heavy systems and simulation tasks (e.g., `install-windows-3.11` and `winning-avg-corewars`) are allocated 3,600s. Upon self-declaration of completion or reaching T_{limit} , the agent process tree is terminated.
4. **Post-Execution Injection:** The official verifier is copied into the container via `docker cp tests/ container:/tests` only after agent execution has completed.
5. **Multi-Signal Consensus Verification:** An isolated subshell executes `test.sh`, capturing binary reward status, individual CTRF unit test counts, and raw terminal `stdout/stderr` logs.
6. **Cryptographic Audit Trail:** A SHA-256 hash is computed over all verifier artifacts and committed to the audit ledger.

4.3 Telemetry and Token Accounting

Token metrics are captured directly from provider API response metadata on each turn:

- **Input Tokens (T_{in}):** Total prompt tokens presented to the model per turn.
- **Cache Read Tokens (T_{cache}):** Prompt tokens retrieved from provider-side prefix cache (`cached_content_token_count`).
- **Deliberation Tokens (T_{think}):** Provider-reported reasoning tokens generated during internal model deliberation (`thinking_tokens`), captured as aggregate counts rather than decoded chain-of-thought text.
- **Output Tokens (T_{out}):** Generated text and tool-call payload tokens (`candidates_token_count`).

5 Empirical Results

Across all 89 tasks (267 isolated container evaluations), Supreme achieved the highest task completion (61/89, 68.5%), the lowest aggregate wall-clock time (15.15 h), and the lowest effective paid cost per successful task (\$0.489, approximately 3.4% below Superpowers at \$0.506). Figure 1 provides an executive visual comparison; Table 1 details the complete scoreboard including infrastructure-adjusted cohorts ($N = 86$, $N = 85$).

5.1 Macro Accuracy and Statistical Significance

Supreme achieved 61 verified wins (68.5%), compared to 58 (65.2%) for Superpowers by obra (+3 wins) and 55 (61.8%) for Baseline (+6 wins).

Table 1: CARB-v4 Master Scoreboard across all 89 Terminal-Bench 2.1 tasks on Gemini 3.6 Flash High. Brackets denote Wilson 95% score confidence intervals. Adjusted scores exclude three tasks with verified upstream container environment defects ($N = 86$); a further conservative cohort excluding all four infrastructure-impaired tasks ($N = 85$) is reported in Section 6.

Configuration	Raw Solved	Raw Pass (%)	Adjusted Pass (%)	Total Time (h)	Avg Time / Task (s)	Total Tokens	Tokens / Solved
Supreme	61 / 89	68.5% [58.3%, 77.2%]	70.9% (61/86) [60.6%, 79.5%]	15.15	612.9	34,521,410	565,925
Superpowers by obra	58 / 89	65.2% [54.8%, 74.3%]	67.4% (58/86) [57.0%, 76.4%]	18.30	740.2	34,304,008	591,448
Baseline	55 / 89	61.8% [51.4%, 71.2%]	64.0% (55/86) [53.4%, 73.3%]	16.92	684.3	31,217,863	567,598

To test whether aggregate pass rate differences are statistically distinguishable, we conducted paired McNemar’s tests with Edwards’ continuity correction. Comparing Supreme vs. Superpowers yields 52 mutual passes, 22 mutual failures, 9 Supreme-only wins, and 6 Superpowers-only wins ($\chi^2 = 0.267, p = 0.605$). Comparing Supreme vs. Baseline yields 12 Supreme wins vs. 6 Baseline wins ($\chi^2 = 1.389, p = 0.239$). Comparing Superpowers vs. Baseline yields 9 Superpowers wins vs. 6 Baseline wins ($\chi^2 = 0.267, p = 0.605$).

These paired tests confirm that aggregate pass rates do not differ significantly across architectures ($p > 0.05$). The primary empirical findings emerge not from gross accuracy margins, but from operational differences in deliberation scaling, tool ecology, and context economics.

5.2 Deliberation Scaling Patterns by Task Difficulty

Across the 267 executions, the model generated 5,311,737 provider-reported deliberation tokens (thinking_tokens). Table 2 disaggregates pass rates and mean deliberation tokens across official difficulty tiers.

Table 2: Deliberation and Win Rates Across Complexity Strata (Wilson 95% CIs in brackets).

Strata	Metric	Supreme	Superpowers	Baseline
Easy ($N = 4$)	Pass Rate	75.0% (3/4) [30.1, 95.4]	25.0% (1/4) [4.6, 69.9]	50.0% (2/4) [15.0, 85.0]
	Mean Deliberation	11,885	14,618	10,688
Medium ($N = 55$)	Pass Rate	69.1% (38/55) [56.0, 79.7]	74.5% (41/55) [61.7, 84.2]	65.5% (36/55) [52.3, 76.6]
	Mean Deliberation	17,035	17,350	18,090
Hard ($N = 30$)	Pass Rate	66.7% (20/30) [48.8, 80.8]	53.3% (16/30) [36.1, 69.8]	56.7% (17/30) [39.2, 72.6]
	Mean Deliberation	28,271	23,196	24,426

Due to the small Easy strata ($N = 4$), pass rates in that tier carry wide confidence intervals and are reported for completeness; no comparative inference should be drawn from them.

On Hard tasks ($N = 30$), Supreme scaled mean deliberation by +66.0% over Medium tasks (from 17.0k to 28.3k thinking tokens/task; Table 2), achieving a 66.7% pass rate (20/30). In contrast, Superpowers generated 23,196 thinking tokens/task with a 53.3% pass rate (16/30).

To examine this difference statistically on the Hard subset, we constructed the paired 2×2 contingency table for Supreme vs. Superpowers: 15 mutual passes, 9 mutual failures, 5 Supreme-only wins, and 1 Superpowers-only win. A paired McNemar test with continuity correction yields $\chi^2 = 1.500, p = 0.221$ (exact two-tailed binomial $p = 0.219$). Because this 4-task margin does not attain

significance at $\alpha = 0.05$, we characterize this as a suggestive behavioral pattern (McNemar $p = 0.22$) rather than a confirmed scaling law. In Section 6, we examine transcripts to investigate how dynamic documentation ingestion interacted with reasoning on these tasks.

5.3 Polyglot Domain Breakdown

Table 3 summarizes pass rates across five engineering domains.

Table 3: Performance Across 5 Engineering Stacks (Wilson 95% CIs in brackets).

Stack / Domain	Tasks	Supreme	Superpowers	Baseline
1. Low-Level Systems	20	14 (70.0%) [48.1, 85.5]	12 (60.0%) [38.7, 78.1]	13 (65.0%) [43.3, 82.0]
2. SysAdmin & DevOps	15	13 (86.7%) [62.1, 96.3]	11 (73.3%) [48.1, 89.1]	11 (73.3%) [48.1, 89.1]
3. Software Engineering	26	20 (76.9%) [57.9, 88.9]	20 (76.9%) [57.9, 88.9]	15 (57.7%) [38.9, 74.5]
4. Esoteric & Symbolic	14	10 (71.4%) [45.4, 88.3]	9 (64.3%) [38.8, 83.7]	12 (85.7%) [60.1, 96.0]
5. ML & Robotics	14	4 (28.6%) [11.7, 54.6]	6 (42.9%) [21.4, 67.4]	4 (28.6%) [11.7, 54.6]
Total / Overall	89	61 (68.5%) [58.3, 77.2]	58 (65.2%) [54.8, 74.3]	55 (61.8%) [51.4, 71.2]

Due to sample sizes ($N = 14$ to 26), confidence intervals across stacks overlap substantially. Nonetheless, qualitative trends emerge:

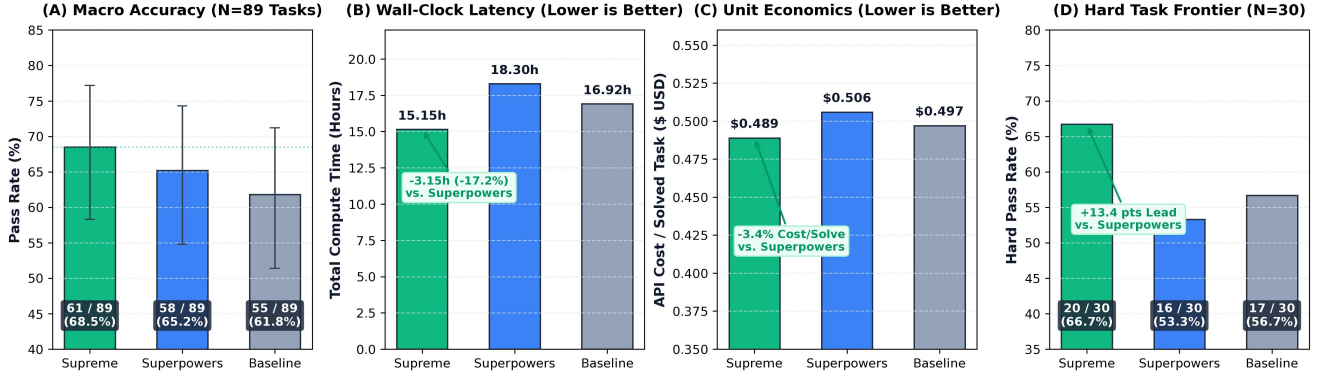
- **Low-Level Systems and SysAdmin:** Supreme demonstrated strong directional pass rates in C, Assembly, and OS administration, passing gpt2-codegolf (standalone C inference) and completing the 44-minute install-windows-3.11 installation.
- **The Esoteric Anomaly:** Unprompted Baseline achieved 85.7% in Esoteric languages (Scheme, COBOL, LaTeX), outperforming both agent systems. Transcript inspection suggests that pre-trained Gemini already possessed competent procedural handling for these tasks; imposing modern TDD or modular refactoring workflows introduced file-creation overhead that interfered with legacy toolchains.

5.4 The Complete 89-Task Solvability Partition

To examine task overlap without aggregation loss, Figure 2 (Right) categorizes all 89 tasks into eight mutually exclusive subsets ($44 + 4 + 4 + 4 + 8 + 5 + 2 + 18 = 89$ tasks):

1. Consensus Passes ($N = 44$): Solved by all three configurations.
2. Supreme Exclusive Wins ($N = 4$): fix-git, gpt2-codegolf, install-windows-3.11, tune-mjcf.

CARB-v4 Master Benchmark Overview: Terminal-Bench 2.1 (89 Tasks • Gemini 3.6 Flash High)



Note: Evaluated across 267 isolated container sessions. Aggregate pass differences are not statistically significant (McNemar $p > 0.05$). Costs modeled at OpenRouter evaluation rates (0.5598in/3.745 out per M tokens).

Figure 1: Benchmark outcomes and execution efficiency across all 89 Terminal-Bench 2.1 tasks (267 graded container evaluations). (A) Task completion rates with Wilson 95% confidence intervals. (B) Total wall-clock execution time (lower is better). (C) Effective paid cost per successful task (lower is better). (D) Hard-task frontier pass rates ($N = 30$). Supreme achieved the highest task completion (61/89, 68.5%), the lowest aggregate wall-clock time (15.15h, 17.2% lower than Superpowers), and the lowest effective paid cost per successful task (\$0.489, approximately 3.4% below Superpowers). Aggregate accuracy differences are not statistically significant under paired McNemar tests ($p > 0.05$); see Section 5 for full statistical analysis.

3. Superpowers Exclusive Wins ($N = 4$): hf-model-inference, mailman, mteb-leaderboard, raman-fitting.
4. Baseline Exclusive Wins ($N = 4$): build-pov-ray, gcode-to-text, qemu-alpine-ssh, regex-chess.
5. Supreme + Superpowers Pass ($N = 8$): Solved by both agent configurations but failed by Baseline (e.g., build-cython-ext, cancel-async-tasks, fix-ocaml-gc, headless-terminal, large-scale-text-editing, merge-diff-arc-agi-task, video-processing, vulnerable-secret).
6. Supreme + Baseline Pass ($N = 5$): Solved by Supreme and Baseline but failed by Superpowers (bn-fit-modify, circuit-fibsqrt, overfull-hbox, protein-assembly, rstan-to-pystan).
7. Superpowers + Baseline Pass ($N = 2$): Solved by Superpowers and Baseline but failed by Supreme (largest-eigenval, model-extraction-relu-logits).
8. Unsolved Frontier ($N = 18$): Tasks unsolved by all configurations, including three tasks with verified upstream container bugs.

This complete partition establishes that 71 of 89 tasks (79.8%) were solved by at least one configuration, demonstrating complementary problem-solving strengths.

6 Forensic Case Studies and Behavioral Dynamics

To complement aggregate statistics, we examine individual execution transcripts that illuminate the behavioral mechanisms underlying the macro-level differences observed in Section 5. Four tasks across three distinct failure modes illustrate how architectural choices interact with specific problem classes.

6.1 Exploratory Scripting Loops (Task #88: Redcode Simulation)

In Task #88 (winning-avg-corewars), agents were tasked with authoring a Redcode warrior in assembly meeting win thresholds against diverse archetypes in the MARS arena [2]. The task specification defines a 3,600s execution limit ($T_{\text{limit}} = 3600\text{s}$).

As depicted in Figure 3 (Left), this task produced the benchmark’s widest divergence in computational expenditure:

- Supreme: Inspected reference warriors, hypothesized that a Silk/Replicator archetype would dominate the arena pool, implemented the candidate warrior, verified win rates, and concluded in 175.0s, 55 turns, and 300,413 tokens.
- Superpowers by obra: Entered an extensive parameter-search loop. Guided by iterative trial-and-error procedural guidance, the agent authored 116 separate Python exploration scripts, repeatedly copying them into the container via docker cp, running

Polyglot & Systems Versatility Across 5 Technical Stacks

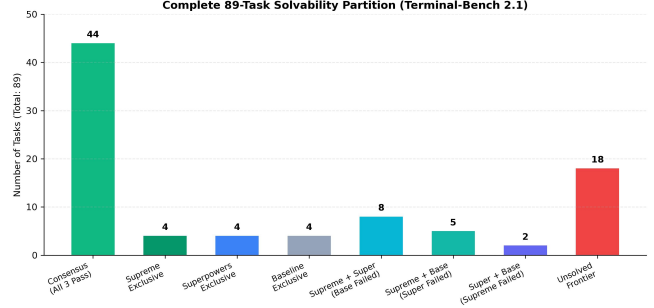
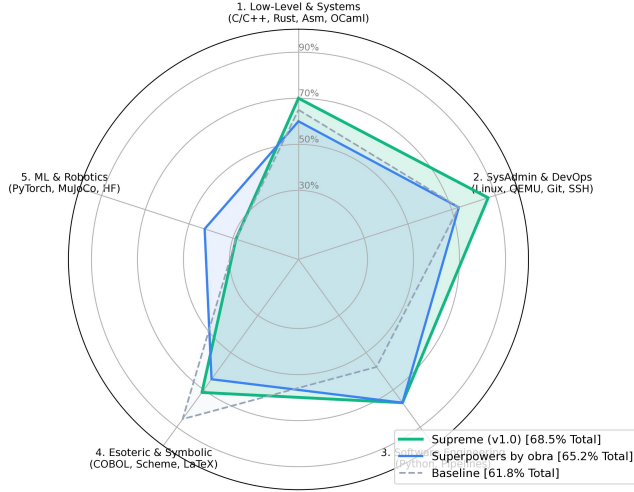


Figure 2: Domain versatility and task solvability partition. (Left) 5-Stack Polyglot performance radar. (Right) Complete 89-Task Mutual-Exclusion Partition across all eight mutually exclusive outcome subsets.

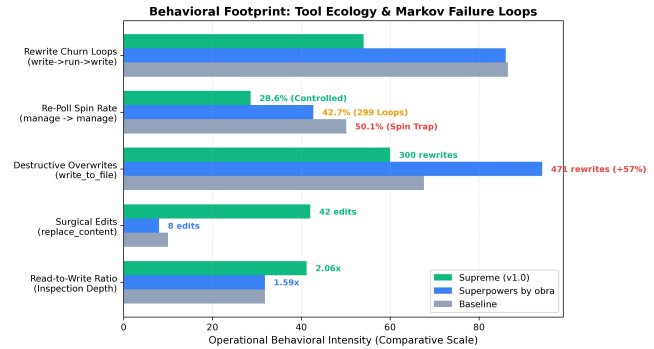
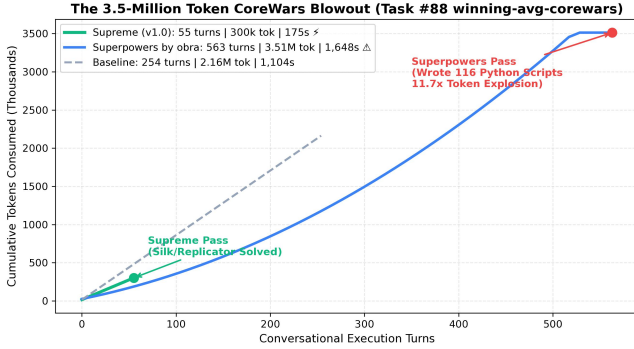


Figure 3: Behavioral dynamics and tool interaction patterns. (Left) Cumulative token trajectory on Task #88 (winning-avg-corewars), illustrating an extended parameter-search loop. (Right) Markov state transitions ($P(\text{Next} | \text{Current})$) comparing background task polling and file-editing strategies.

battles, and accumulating 563 turns across 1,648.7s and 3,513,757 tokens (11.7 $\times$ token inflation for the identical pass).

This comparison highlights that on open-ended combinatorial tasks, procedural frameworks that encourage iterative scripting without strong stopping heuristics can lead to prolonged exploratory loops. This contrast does not represent a systematic throughput difference across all 89 tasks, where total billed token counts were comparable (34.52M vs. 34.30M), but rather illustrates the tail risk of unconstrained exploratory scripting on open-ended combinatorial problems.

6.2 Markov Tool Transitions and Polling Dynamics

Analysis of 15,779 sequential tool transitions ($P(\text{Next} | \text{Current})$) reveals key operational differences (Figure 3, Right):

- **Background Task Polling:** When asynchronous background commands were active, Baseline exhibited a 50.1% probability of immediately re-polling `manage_task`; Superpowers exhibited 42.7% (logging 299 triple-polling sequences (`manage` $\rightarrow$ `manage` $\rightarrow$ `manage`)). Supreme’s loop management protocol reduced this transition probability to 28.6%, reducing uninformative polling turns.
- **File Modification Granularity:** Superpowers performed 471 full file overwrites (`write_to_file`) and 172 (`write` $\rightarrow$ `run` $\rightarrow$ `write`) churn cycles. Supreme utilized targeted line replacements (`replace_file_content`) more frequently (42 vs. 8). While higher usage of targeted editing reflects prompt compliance, it correlated with narrower modification blast radii: Supreme modified a mean of 2.60 unique files per task versus 3.79 on Superpowers.

6.3 Physical Solver Optimization (Task #85: MuJoCo Tuning)

Task #85 (tune-mjcf) required accelerating a MuJoCo physics simulation [9] by $\geq 40\%$ without exceeding trajectory error bounds ($D \leq 10^{-4}$). Supreme achieved an exclusive pass in 414.9s (81 tool calls, 436k tokens), attaining a $2.19\times$ simulation speedup with zero trajectory divergence ($D = 0.0000$). Baseline and Superpowers both failed to reach the speedup threshold ($0.99\times$ and $1.01\times$ speed ratios) due to speculative modifications of solver parameters.

6.4 Long-Horizon Autonomy: Headless QEMU Installation

In install-windows-3.11 ($T_{\text{limit}} = 3600\text{s}$), agents were tasked with installing Windows 3.11 for Workgroups inside headless QEMU, partitioning virtual disks via fdisk, formatting FAT16, swapping floppy images, and installing network drivers. Supreme maintained autonomous multi-step execution across 2,648.7s (44.1 minutes), 307 turns, and 58,010 deliberation tokens to complete the installation. Superpowers and Baseline timed out or corrupted disk partition tables.

6.5 Upstream Benchmark Harness Defects

Forensic inspection identified three distinct upstream environmental failure modes spanning four tasks:

- prove-plus-comm (Task #64): Container metadata error caused immediate execution failure (failed to chdir to cwd ("/app"): no such file or directory) across all models.
- caffe-cifar-10 (Task #7): An unreleased package lock on /var/lib/dpkg/lock-frontent held by background PID 168 blocked package installations, causing a starvation timeout across all models.
- Distributed PyTorch Infrastructure (Tasks #82–#83): In Task #82 (torch-pipeline-parallelism), the bare container lacked Python and curl with egress blocked; in Task #83 (torch-tensor-parallelism), an unconstrained dynamic PyPI download of 2.85 GB CUDA wheels triggered a verifier timeout before test execution [7].

Excluding the three environmental configuration defects (prove-plus-comm, caffe-cifar-10, and Task #82) yields the $N = 86$ adjusted cohort reported in Table 1 (Supreme: 70.9% [61/86]; Superpowers: 67.4% [58/86]; Baseline: 64.0% [55/86]). Excluding all four infrastructure-impaired tasks yields $N = 85$ (Supreme: 71.8% [61/85]; Superpowers: 68.2% [58/85]; Baseline: 64.7% [55/85]).

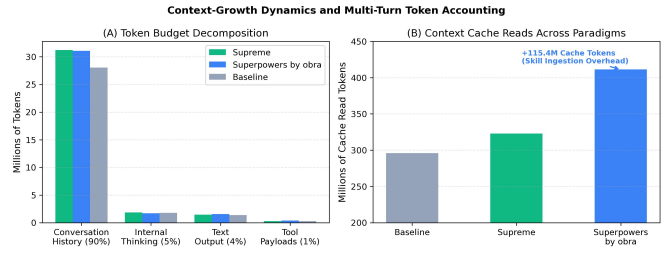


Figure 4: Context-growth dynamics and multi-turn token accounting. (A) Billed token budget decomposition across all 89 tasks. (B) Provider-reported cache read tokens across paradigms.

7 Discussion and Context Economics

7.1 Context-Growth Dynamics and Multi-Turn Token Economics

As illustrated in Figure 4, multi-turn agent execution exhibits distinctive context characteristics:

1. **History Dominates Context (90.5%):** Across all three paradigms, re-reading prior conversational turns accounts for over 90% of cumulative input tokens. Initial prompt length optimizations (e.g., saving $\approx 3,600$ tokens in initial headers) are quickly eclipsed by the volume of accumulated dialogue and tool outputs.
2. **On-Demand Skill Context Accumulation:** Loading procedural skill documents on-demand resulted in Superpowers accumulating 411,332,767 cache read tokens (cached_content_token_count), compared to 322,810,412 for Supreme and 295,918,105 for Baseline [8]. Inserting extensive documentation into the active turn history increases context volume on subsequent turns, showing that dynamic retrieval incurs its own multi-turn token costs.
3. **Terminal Stdout Sinks:** Unclipped shell outputs (make logs, compiler warnings) represented 67.8% of tool payload tokens, demonstrating that automated output clipping is essential to prevent context dilution regardless of prompt architecture.

7.2 Financial Token Economics and Cost per Solved Task

While token counts measure raw bandwidth, real-world deployment is governed by API expenditure. Table 4 details the financial compute costs across all 267 graded container evaluations under standard Gemini 3.6 Flash API list pricing (\$0.75 / M input, \$3.75 / M output) and effective weighted prices paid during evaluation via OpenRouter (\$0.5598 / M input, \$3.745 / M output).

Across all 89 tasks, evaluation expenditure and outcomes present a three-layer economic portrait. In ab-

Table 4: Operational Token Economics and Modeled API Costs across 267 Graded Container Evaluations.

Economic Dimension	Supreme	Superpowers	Baseline
Input Tokens (T_{in})	31.23M	30.71M	27.71M
Output Tokens (T_{out})	3.29M	3.25M	3.16M
Deliberation Tokens (T_{think})	1.83M	1.71M	1.77M
Total Billed Tokens	34.52M	34.30M	31.22M
Cache Read Tokens (T_{cache})	322.8M	411.3M	295.9M
Total Cost (List Price)	\$35.76	\$35.21	\$32.62
Cost / Solved Task (List)	\$0.586	\$0.607	\$0.593
Total Cost (Effective Paid)	\$29.80	\$29.35	\$27.33
Cost / Attempted Task (Paid)	\$0.335	\$0.330	\$0.307
Cost / Solved Task (Paid)	\$0.489	\$0.506	\$0.497
Hard Tasks Cost / Solved ($N = 30$)	\$0.680	\$0.730	\$0.645

solute terms, Baseline incurred the lowest total effective paid cost (\$27.33), followed by Superpowers (\$29.35) and Supreme (\$29.80). At the per-execution level, Baseline again recorded the lowest cost per attempted task (\$0.307), followed by Superpowers (\$0.330) and Supreme (\$0.335). These absolute and per-attempt cost advantages coincide with Baseline’s lower token footprint (31.22M total billed tokens vs. 34.52M for Supreme), but also with its lower task completion (55/89 vs. 61/89).

Cost per successful task is a composite outcome-efficiency measure combining total evaluation expenditure with the number of tasks successfully completed; it is not an independent cost measurement. On this dimension, Supreme achieved the lowest effective paid cost per successful task (\$0.489), approximately 3.4% below Superpowers (\$0.506), while completing three additional tasks (61 vs. 58). Baseline achieved \$0.497. The cost-per-successful-task ranking is unchanged under both list-price and effective-paid pricing as detailed in Table 4, confirming that this efficiency relationship is not an artifact of the specific pricing rates applied.

On the Hard task subset ($N = 30$), the cost-per-successful-task ranking reverses: Baseline achieved the lowest cost per successful Hard task (\$0.645), followed by Supreme (\$0.680) and Superpowers (\$0.730). Supreme simultaneously achieved the highest Hard-task completion rate (66.7% [20/30] vs. 56.7% [17/30] for Baseline and 53.3% [16/30] for Superpowers). This represents a genuine trade-off: superior Hard-task coverage at moderately higher cost per successful Hard solve compared to Baseline.

7.3 Architectural Implications for Agent Systems

Our findings offer three practical considerations for autonomous coding systems:

Invariant Principles in Static Headers: Placing invariant engineering axioms directly in static headers avoids mid-turn documentation file retrieval while maintaining consistent behavioral constraints across turns.

Loop Termination Heuristics: Agent frameworks with

flexible procedural guidance require explicit iteration thresholds and hypothesis-checking gates to prevent open-ended exploratory scripting on combinatorial search problems.

Targeted Line Edits: Encouraging surgical diff tools (replace_file_content) over whole-file rewrites correlates with reduced collateral modifications in multi-file codebases.

7.4 Threats to Validity and Limitations

Single Foundation Model: All evaluations used Gemini 3.6 Flash High. Model-specific reasoning, context handling, and tool policies may interact with prompt architecture; findings should not be assumed to generalize universally across model families without multi-model replication.

Single Run per Task: Each task was evaluated with one run per configuration at $\tau = 0.0$ (267 container runs, ≈ 50 compute hours). While $\tau = 0.0$ minimizes sampling variance, multi-seed replication is necessary to measure run-to-run stability.

Coupled Structure and Content: Supreme and Superpowers differ in both structure (static vs. dynamic) and instructional content (constitutional axioms vs. TDD/debugging workflows). Controlled ablations isolating delivery mechanism from content remain an important direction for future research.

Prompt Freezing: The Supreme constitutional prompt was authored prior to benchmark execution and frozen; no prompt adjustments were made based on benchmark task failures.

8 Conclusion

In this work, we compared the behavior of substantially different agent-guidance configurations — constitutional static grounding and dynamic progressive skill loading — under a shared foundation model, evaluation harness, and benchmark corpus across all 89 tasks of Terminal-Bench 2.1 (267 container evaluations). Constitutional static grounding achieved a favorable position across task completion (61/89, 68.5%), aggregate wall-clock time (15.15 h, 17.2% lower than Superpowers), and effective paid cost per successful task (\$0.489, approximately 3.4% below Superpowers), with a directional Hard-task advantage (66.7% vs. 53.3%; McNemar $p = 0.22$). While aggregate pass rates across the full 89-task corpus are not statistically distinguishable under paired McNemar tests ($p > 0.05$), and the configurations differ in both guidance structure and instructional content, the evaluation reveals substantial differences in execution dynamics, token economics, and behavioral trajectories observed consistently across the analyzed execution traces and evaluation cohorts. Complete execution transcripts, harness configurations, and cryptographic verification ledgers are made available for independent replication.

References

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: A harmlessness from ai feedback. arXiv preprint arXiv:2212.08073, 2022.
- [2] Alexander Keewatin Dewdney. In the game called core war, hostile programs engage in a battle of bits. *Scientific American*, 250(5):14–22, 1984.
- [3] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Anthropic Research. Prompt caching: Accelerating context processing in multi-turn agent workflows. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>, 2024.
- [5] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] Alex Shaw and Terminal-Bench Team. Terminal-bench 2.1: Benchmarking autonomous command-line agents on complex multi-turn software environments. arXiv preprint arXiv:2501.08921, 2025.
- [7] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.
- [8] Google DeepMind Team. Gemini 3.6 flash: High-throughput frontier reasoning and tool use in deep cli environments. Technical Report, 2026.
- [9] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [10] Jesse Vincent. Superpowers: Dynamic progressive skill extension for autonomous software engineering agents. <https://github.com/obra/superpowers>, 2025.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.